Spring Core & Maven — Hands-on Exercises

Library Management Application

Exercise 1: Configuring a Basic Spring Application

Basic idea here is to get a Maven project going and let Spring wire up the BookService and BookRepository beans through an XML config file instead of creating objects with 'new' everywhere.

1. pom.xml — Spring Core dependency

```
<dependencies>
   <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-context</artifactId>
      <version>5.3.30</version>
   </dependency>
</dependencies>
```

2. src/main/resources/applicationContext.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans.xsd">

   <bean id="bookRepository" class="com.library.repository.BookRepository" />

   <bean id="bookService" class="com.library.service.BookService" />

</beans>
```

3. Service and Repository classes

com/library/repository/BookRepository.java

```
package com.library.repository;

public class BookRepository {
   public void addBook(String bookName) {
      System.out.println("Book added to repository: " + bookName);
   }
}
```

com/library/service/BookService.java

```
package com.library.service;

public class BookService {
   public void addBook(String bookName) {
      System.out.println("Service layer processing book: " + bookName);
   }
}
```

4. Main class to load the context

```
package com.library;
```

```
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.library.service.BookService;

public class LibraryManagementApplication {
    public static void main(String[] args) {
        ApplicationContext context =
                new ClassPathXmlApplicationContext("applicationContext.xml");
        BookService bookService = (BookService) context.getBean("bookService");
        bookService.addBook("Java Fundamentals");
    }
}
```

Output: "Service layer processing book: Java Fundamentals" — confirms the context loaded and the bean got created by Spring, not by me.

Exercise 2: Implementing Dependency Injection

Now BookService actually needs BookRepository to do anything useful, so this is about wiring them together with setter injection instead of hardcoding the dependency.

1. Updated applicationContext.xml

```
<bean id="bookRepository" class="com.library.repository.BookRepository" />

<bean id="bookService" class="com.library.service.BookService">
    <property name="bookRepository" ref="bookRepository" />
</bean>
```

2. BookService with a setter method

```
package com.library.service;

import com.library.repository.BookRepository;

public class BookService {

    private BookRepository bookRepository;

    public void setBookRepository(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public void addBook(String bookName) {
        bookRepository.addBook(bookName);
    }
}
```

3. Testing

Running LibraryManagementApplication now prints "Book added to repository: Java Fundamentals" — that message only comes from BookRepository, so it proves Spring injected the dependency instead of BookService talking to itself.

Exercise 3: Implementing Logging with Spring AOP

Here the goal is to log how long a method takes without touching the actual business logic inside BookService — that's the whole point of AOP, keeping the cross-cutting stuff separate.

1. pom.xml — add AOP + AspectJ weaver

```
<dependency>
   <groupId>org.springframework</groupId>
   <artifactId>spring-aop</artifactId>
   <version>5.3.30</version>
</dependency>
<dependency>
   <groupId>org.aspectj</groupId>
   <artifactId>aspectjweaver</artifactId>
   <version>1.9.19</version>
</dependency>
```

2. com/library/aspect/LoggingAspect.java

```
package com.library.aspect;

import org.aspectj.lang.ProceedingJoinPoint;

public class LoggingAspect {

   public Object logExecutionTime(ProceedingJoinPoint joinPoint) throws Throwable {
      long start = System.currentTimeMillis();
      Object result = joinPoint.proceed();
      long end = System.currentTimeMillis();
      System.out.println(joinPoint.getSignature() + " took " + (end - start) + " ms");
      return result;
   }
}
```

3. applicationContext.xml — enable AspectJ support

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
      xmlns:aop="http://www.springframework.org/schema/aop"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://www.springframework.org/schema/beans
      http://www.springframework.org/schema/beans/spring-beans.xsd
      http://www.springframework.org/schema/aop
      http://www.springframework.org/schema/aop/spring-aop.xsd">

   <bean id="bookRepository" class="com.library.repository.BookRepository" />
   <bean id="bookService" class="com.library.service.BookService">
      <property name="bookRepository" ref="bookRepository" />
   </bean>
   <bean id="loggingAspect" class="com.library.aspect.LoggingAspect" />

   <aop:config>
      <aop:aspect ref="loggingAspect">
         <aop:pointcut id="serviceMethods"
            expression="execution(* com.library.service.*.*(..))" />
         <aop:around pointcut-ref="serviceMethods" method="logExecutionTime" />
      </aop:aspect>
   </aop:config>

</beans>
```

4. Testing

Running the main class now prints the usual repository message plus something like "void com.library.service.BookService.addBook(String) took 2 ms" — that second line is coming from the aspect, not from BookService itself.

Exercise 4: Creating and Configuring a Maven Project

This one is basically pulling everything from before into a proper standalone Maven project (LibraryManagement) with all three Spring dependencies and the compiler plugin set to Java 8.

Full pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.library</groupId>
  <artifactId>LibraryManagement</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>jar</packaging>

  <properties>
    <maven.compiler.source>1.8</maven.compiler.source>
    <maven.compiler.target>1.8</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-context</artifactId>
      <version>5.3.30</version>
    </dependency>
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-aop</artifactId>
      <version>5.3.30</version>
    </dependency>
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-webmvc</artifactId>
      <version>5.3.30</version>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.11.0</version>
        <configuration>
          <source>1.8</source>
          <target>1.8</target>
        </configuration>
      </plugin>
```

```
    </plugins>
  </build>
</project>
```

Note: I kept maven.compiler.source/target AND the compiler plugin config — either alone is enough in practice, but the exercise specifically asks for the plugin to be configured.

Exercise 5: Configuring the Spring IoC Container

This is really the same central-configuration idea as Exercise 1/2 — one applicationContext.xml file that owns the bean definitions and wiring so nothing gets created manually with 'new' inside the app.

applicationContext.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans.xsd">

   <bean id="bookRepository" class="com.library.repository.BookRepository" />

   <bean id="bookService" class="com.library.service.BookService">
      <property name="bookRepository" ref="bookRepository" />
   </bean>

</beans>
```

BookService keeps the setBookRepository() setter from Exercise 2, and the same LibraryManagementApplication main class is used to load the context and call bookService.addBook(...) to confirm the container is wired correctly.

Exercise 6: Configuring Beans with Annotations

Instead of declaring every bean by hand in XML, this swaps to component scanning plus @Service / @Repository so Spring finds and registers the beans on its own.

1. applicationContext.xml — component scan

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:context="http://www.springframework.org/schema/context"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans.xsd
       http://www.springframework.org/schema/context
       http://www.springframework.org/schema/context/spring-context.xsd">

   <context:component-scan base-package="com.library" />

</beans>
```

2. Annotated classes

com/library/repository/BookRepository.java

```
package com.library.repository;
```

```
import org.springframework.stereotype.Repository;

@Repository
public class BookRepository {
    public void addBook(String bookName) {
        System.out.println("Book added to repository: " + bookName);
    }
}
```

com/library/service/BookService.java

```
package com.library.service;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import com.library.repository.BookRepository;

@Service
public class BookService {

    @Autowired
    private BookRepository bookRepository;

    public void addBook(String bookName) {
        bookRepository.addBook(bookName);
    }
}
```

3. Testing

Same main class as before, no changes needed there — running it still prints the repository message, just this time the beans got picked up via @Service/@Repository scanning instead of explicit <bean> tags.

Exercise 7: Implementing Constructor and Setter Injection

This exercise wants both injection styles configured together so BookService can be built either way — through the constructor or through the setter.

1. applicationContext.xml

```
<bean id="bookRepository" class="com.library.repository.BookRepository" />

<bean id="bookService" class="com.library.service.BookService">
    <constructor-arg ref="bookRepository" />
    <property name="bookRepository" ref="bookRepository" />
</bean>
```

2. BookService with both a constructor and a setter

```
package com.library.service;

import com.library.repository.BookRepository;

public class BookService {

    private BookRepository bookRepository;

    public BookService(BookRepository bookRepository) {
```

```
        this.bookRepository = bookRepository;
    }

    public void setBookRepository(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public void addBook(String bookName) {
        bookRepository.addBook(bookName);
    }
}
```

3. Testing

Running LibraryManagementApplication still gives the same repository-added message, confirming bookRepository got set — in a real project I'd only use one of the two injection styles, but the exercise specifically asks to demonstrate both.

Exercise 8: Implementing Basic AOP with Spring

This is a simpler AOP setup compared to Exercise 3 — just plain before/after advice around the service methods, no timing calculation involved.

1. com/library/aspect/LoggingAspect.java

```
package com.library.aspect;

public class LoggingAspect {

    public void beforeMethod() {
        System.out.println("LOG: method execution started");
    }

    public void afterMethod() {
        System.out.println("LOG: method execution finished");
    }
}
```

2. applicationContext.xml — register the aspect

```
<bean id="loggingAspect" class="com.library.aspect.LoggingAspect" />

<aop:config>
    <aop:aspect ref="loggingAspect">
        <aop:pointcut id="serviceMethods"
            expression="execution(* com.library.service.*.*(..))" />
        <aop:before pointcut-ref="serviceMethods" method="beforeMethod" />
        <aop:after pointcut-ref="serviceMethods" method="afterMethod" />
    </aop:aspect>
</aop:config>

<aop:aspectj-autoproxy />
```

3. Testing

Console output on running the app: "LOG: method execution started", then the repository message, then "LOG: method execution finished" — shows the advice is wrapping around addBook() correctly.

Exercise 9: Creating a Spring Boot Application

Last one moves away from XML config entirely and uses Spring Boot with an in-memory H2 database and a REST controller for actual CRUD operations on books.

1. Dependencies (via Spring Initializr) — pom.xml

```
<dependencies>
   <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
   </dependency>
   <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
   </dependency>
   <dependency>
      <groupId>com.h2database</groupId>
      <artifactId>h2</artifactId>
      <scope>runtime</scope>
   </dependency>
</dependencies>
```

2. src/main/resources/application.properties

```
spring.datasource.url=jdbc:h2:mem:librarydb
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.h2.console.enabled=true
```

3. Entity and Repository

com/library/model/Book.java

```
package com.library.model;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class Book {

   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;
   private String title;
   private String author;

   public Long getId() { return id; }
   public void setId(Long id) { this.id = id; }
   public String getTitle() { return title; }
   public void setTitle(String title) { this.title = title; }
   public String getAuthor() { return author; }
   public void setAuthor(String author) { this.author = author; }
}
```

com/library/repository/BookRepository.java

```
package com.library.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import com.library.model.Book;

public interface BookRepository extends JpaRepository<Book, Long> {
}
```

4. REST Controller

```
package com.library.controller;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import com.library.model.Book;
import com.library.repository.BookRepository;

import java.util.List;

@RestController
@RequestMapping("/books")
public class BookController {

    @Autowired
    private BookRepository bookRepository;

    @GetMapping
    public List<Book> getAllBooks() {
        return bookRepository.findAll();
    }

    @PostMapping
    public Book addBook(@RequestBody Book book) {
        return bookRepository.save(book);
    }

    @GetMapping("/{id}")
    public Book getBook(@PathVariable Long id) {
        return bookRepository.findById(id).orElse(null);
    }

    @PutMapping("/{id}")
    public Book updateBook(@PathVariable Long id, @RequestBody Book book) {
        book.setId(id);
        return bookRepository.save(book);
    }

    @DeleteMapping("/{id}")
    public void deleteBook(@PathVariable Long id) {
        bookRepository.deleteById(id);
    }
}
```

5. Main class

```
package com.library;

import org.springframework.boot.SpringApplication;
```

```
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class LibraryManagementApplication {
    public static void main(String[] args) {
        SpringApplication.run(LibraryManagementApplication.class, args);
    }
}
```

6. Testing the endpoints

Once the app is running on port 8080, I tested it with:

```
curl -X POST http://localhost:8080/books \
  -H "Content-Type: application/json" \
  -d '{"title":"Clean Code","author":"Robert C. Martin"}'

curl http://localhost:8080/books
```

The POST returns the saved book with an auto-generated id, and the GET lists it back — confirms the JPA repository and H2 database are actually working end to end.